Search Algorithms

CSC 480: Artificial Intelligence, Spring 2026

Abstract

I Part 1: Intelligent Agents

Q1: State the definition of a rational agent.

" A rational agent is an agent for which, for each possible percept sequence, it should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has."

Q2: List the dimensions used to characterize environments, giving both sides of each dimension. [1] check the textbook :P

[1] AIMA Chapter 2.3 Page 61

Q3: Explain the difference between a reflex agent and a goal based agent. Why can't a reflex agent solve a maze?

A reflex agent selects actions based only on the current percept (or current percept + simple condition action rules). It has no internal model of the world and no concept of future consequences. A goal based agent has explicit goals and uses search or planning to determine which sequence of actions will achieve them. So a reflex agent will loop infinitely since it has no memory of visited states

Q4: What is the difference between a model based agent and a utility based agent? [2] check the textbook

[2] AIMA Chapter 2.4 Page 69

Contents

I Part 1: Intelligent Agents 1
II Part 2: Search Fundamentals 3
III Part 3: Search with Heuristics . . 7
IV Part 4: Adversarial Search 14

Q5: Classify each of the following environments along dimensions used to characterize environments.

(a) A chess game against a human opponent

- Fully observable (you see the whole board)
- Multi-agent (you and opponent)
- Deterministic (moves have predictable outcomes)
- Sequential (current moves affect future positions)
- Static (board doesn't change while you think (assuming no clock pressure))
- Discrete (finite number of squares, pieces, and moves)
- Known (rules of chess are fully known)

(b) A self driving car on a highway

- Partially observable (sensors have blind spots, cant see intentions of other drivers)
- Multi-agent (other drivers)
- Stochastic (other drivers and conditions are unpredictable)
- Sequential (decisions now affect future trajectory)
- Dynamic (the world keeps moving while the car deliberates)
- Continuous (positions, speeds, and steering angles are continuous)
- Known(rules of the road are but with unknown agent behaviors)

(c) A thermostat in a room

- Partially observable (knows current temperature, not heat sources)
- Single-agent
- Mostly deterministic (or mildly stochastic depending on disturbances)
- Sequential (though often modeled episodically)
- Dynamic (temperature changes over time)
- Continuous (temperature values)
- Known (heat modeling is well understood (i think))

II Part 2: Search Fundamentals

Q6: List and explain the components that formally define a search problem. [3]

[3] AIMA Chapter 3.1 Page 83

The six components are:

(1) States which are the set of all possible configurations the problem can be in (the state space).

(2) Initial state which is the starting configuration the agent begins in.

(3) Goal state or the target configuration (or a goal test that determines whether a given state satisfies the goal).

(4) Actions. The set of possible moves available to the agent in each state.

(5) Transition model which defines the result of applying an action in a state, mapping (state, action) ==> successor state.

(6) Cost function which assigns a numeric cost to each action or path, used to evaluate the expense of reaching the goal.

Q7: For each of the following search strategies, state whether it is complete and whether it is optimal. (Assume a finite branching factor and non-negative edge costs.)

Strategy	**Complete?**	**Optimal?**
Depth-First Search (Graph)	Yes (finite state space)	no
Breadth-First Search (Graph)	Yes (finite branching)	Only if step costs are equal
Uniform Cost Search (Graph)	Yes (if step costs > 0)	Yes
Astar Tree Search (admissible h)	Yes	Yes
Astar Graph Search (consistent h)	Yes	Yes

Q8: In your own words, explain why Breadth First Search is optimal only when all step costs are equal, whereas Uniform Cost Search is optimal for any non-negative step costs.

BFS expands nodes in increments of height/level. It explores the shallowest areas first. If all step costs are equal, the shallowest path is also the cheapest. But if step costs vary, a deeper path might have lower total cost. UCS expands nodes in order of cumulative path cost, so it always expands the cheapest cost node next, guaranteeing it finds the lowest cost path regardless of how costs are distributed.

Q9: Explain the difference between tree search and graph search. Why might tree search expand the same state more than once, and what mechanism does graph search use to prevent this? [4]

Tree search does not track which states have been visited, so it may expand a state multiple times if it appears via different paths in the search tree. Graph search maintains a closed set (or explored set) of all previously expanded states and skips any state that has already been expanded. This prevents redundant exploration but requires additional memory proportional to the number of explored states.

[4] AIMA Chapter 3.3 Page 92

Q10: Consider the following graph. All edges are undirected. The start state is S and the goal state is G.

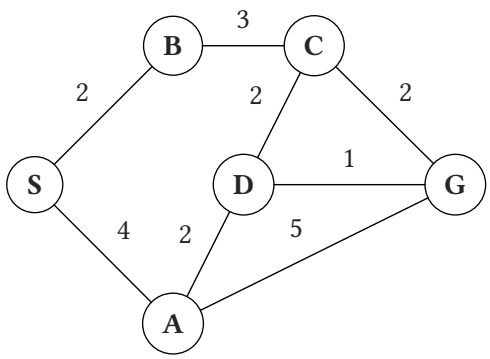

Figure 1: Undirected weighted graph for Q10.

(a) Give the order in which nodes are expanded using Breadth First Search. Break ties alphabetically.

S, A, B, D, G

(b) Give the order in which nodes are expanded using Depth First Search. Break ties alphabetically.

S, A, D, C, B, G

(C) Give the order in which nodes are expanded using Uniform Cost Search. Break ties alphabetically.

S, B, A, C, D, G

Q11: For each search algorithm (DFS, BFS, UCS), state its time complexity and space complexity in terms of branching factor b, solution depth d, and maximum depth m.

Algorithm	**Time**	**Space**
DFS	$O(b^m)$	$O(bm)$
BFS	$O(b^d)$	$O(b^d)$
UCS	$O(b^{1+\lceil C^*/\varepsilon \rceil}) \approx O(b^d)$	$O(b^{1+\lceil C^*/\varepsilon \rceil}) \approx O(b^d)$

Where b = branching factor, d = depth of shallowest solution, m = maximum depth, C^* = cost of optimal solution, and ε = minimum step cost.

Q12: Which algorithm is most memory efficient, and why?

DFS is the most memory efficient. It only needs to store the current path from root to the deepest expanded node, plus the unexpanded siblings along that path. Which results in O(bm) space complexity (linear in depth). BFS and UCS, however, must store the entire frontier, which grows exponentially with depth (O(b^d)). For deep search problems where memory is the bottleneck, DFS or its variants (like iterative deepening[5]) are the only practical options.

[5] AIMA Chapter 3.4.4 Page 98

III Part 3: Search with Heuristics

Q13: State the two properties a heuristic must satisfy to be consistent. How does consistency relate to admissibility? [6]

[6] AIMA Chapter 3.5 Page 106

check the textbook

Q14: A robot navigates a grid. Its state is represented as (row, col). The start is (0, 0) and the goal is (3, 4). The robot can move up, down, left, or right with cost 1 per step.

(a) What is the value of the Manhattan distance heuristic from the goal to the start (ie: h(start))?

Manhattan distance = $|3 - 0| + |4 - 0| = 3 + 4 = \mathbf{7}$.

(b) Suppose an obstacle blocks the direct path so the shortest actual path has cost 9. Does A-star with the Manhattan distance heuristic still find the optimal path? Why or why not?

The Manhattan distance is admissible. Astar with an admissible heuristic is guaranteed to find the optimal path, even though the optimal cost (9) exceeds the heuristic estimate (7).

Astar will simply expand more nodes before finding the goal. The admissibility guarantee will hold regardless of how loose the estimate is, as long as it never overestimates.

Q15: Using the same robot and grid from Q14: A heuristic h1 returns 0 for every state. A heuristic h2 returns the Manhattan distance.

(a) Are both admissible? Explain your answer. [7]

[7] AIMA Chapter 3.5 Page 104

h1 = 0 is trivially admissible[8]. h2 = Manhattan distance is admissible for grid movement with cost 1 per step, since any path between two points must traverse at least the Manhattan distance in steps.

[8] 0 <= hstar(n) for all n, since path costs are non-negative

(b) Which one causes A-star to expand fewer nodes, and why?

h2 causes Astar to expand fewer nodes. A more informative[9] heuristic focuses the search more tightly toward the goal by giving distant nodes higher f values, deprioritizing them. h1 = 0 makes Astar behave identically to UCS, expanding nodes uniformly outward in all directions with no goal directed steering.

[9] higher valued, still admissible

(c) If we define h3(n) = max(h1(n), h2(n)) (using h1 and h2 from Q15), is h3 admissible? Is it at least as informative as h1 and h2 individually?

h3 = max(h1, h2) = max(0, Manhattan) = Manhattan.

Since both are admissible, the max of two admissible heuristics is also admissible. h3 dominates both h1 and h2 because by definition h3(n) >= h1(n) and h3(n) >= h2(n) for all n.

Q16: Jeffrey Armstrong claims: "If a heuristic is inadmissible, A-star will never find the optimal solution." Is this claim always true, sometimes true, or always false? Provide a brief justification or counterexample.

Sometimes true. [10]

Counterexample: Consider a graph where the heuristic overestimates only at states that arent on the optimal path. Astar may still expand the optimal path first and return it. The point of admissibility is to guarantee optimality. So losing admissibility means losing the guarantee, not necessarily losing optimality on every instance.

[10] An inadmissible heuristic can still find the optimal solution, however it just isnt guaranteed to.

Q17: Explain why A-star with a consistent heuristic guarantees that when a node is first expanded in graph search, it has been reached by the optimal path. Why does admissibility alone not guarantee this for A-star graph search?

Consistency ensures that f values along any path are non-decreasing[11]. This means once a node is expanded, no later path to that same state can have a lower f value, so the first expansion is always optimal.

[11] f(n') = g(n') + h(n') >= g(n) + h(n) = f(n)

With merely admissible (but inconsistent) heuristics, f values can decrease along a path. This means a node might be reached via a suboptimal path first (with lower f), get added to the closed set, and then later when a better path arrives, graph search refuses to reexpand it.

Soooo, Astar graph search with an admissible but inconsistent heuristic can return suboptimal solutions![12]

[12] To fix this, you can either use tree search, allow reexpansion when a better path is found, or insist on consistency.

Q18: Jeffery has once again proposed a new heuristic for A-star search that is very accurate but takes O(n^2) time to compute, where n is the problem size. The current heuristic is less accurate but computes in O(1). Under what circumstances would you recommend switching to the more expensive heuristic?

Total runtime for A-star is roughly: (nodes expanded) x (cost per node). The cost per node includes heuristic evaluation, so switching makes sense only when the reduction in nodes expanded outweighs the higher per node cost.

One would switch to the O(n^2) heuristic when:

- The branching factor is large. A more informed heuristic prunes far more of the search tree when b is high, since the frontier grows exponentially. The node savings can exceed the constant overhead of a worse heuristic.
- The solution depth is large. The deeper the goal, the more nodes the inaccurate heuristic explores, so the accurate one pays off more over time.
- n is small. If the problem size n is small, O(n^2) vs O(1) per node is a minor difference in absolute time.

But you would stay with the O(1) heuristic when:

- n is large. If n is large, the O(n^2) evaluation becomes expensive per node and the heuristic cost starts to dominate total runtime.
- The state space is small or shallow. If both heuristics lead to similar node counts, paying O(n^2) per node gains nothing.

Q19: The 15-puzzle is a sliding tile game in which one tries to arrange all the tiles in ascending order(See figure 2 below). Two heuristics are proposed for the 15-puzzle :

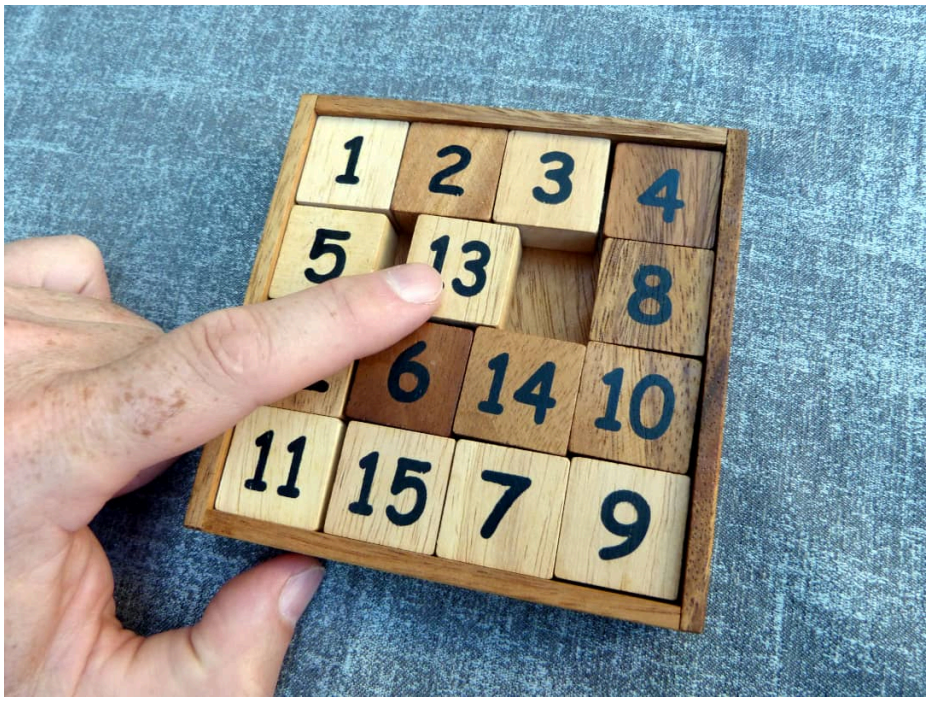

Figure 2: An Example of the 15 puzzle

h1 = number of misplaced tiles

h2 = sum of Manhattan distances of each tile from its goal position

(a) Prove both are admissible.

h1: Each misplaced tile must be moved at least once to reach its goal position and 1 move only changes the position of 1 tile. So the number of moves required is at least the number of misplaced tiles. h1(n) <= hstar(n).

h2: Each tile must travel at least its manhattan distance to its goal position. Each move shifts a single tile by exactly 1 in manhattan distance. So the total number of moves is at least the sum of Manhattan distances. h2(n) <= hstar(n).

(b) Prove that h2 dominates h1 (h2(n) >= h1(n) for all n). What does dominance guarantee about the number of nodes expanded?

for each of the misplaced tile the manhattan distance is at least 1 (since its not in its goal/finish square). So each tile contributes at least 1 to h2 and exactly 1 to h1. For any tile thats farther than 1 square from its goal it contributes more to h2 than to h1. Therefore h2(n) >= h1(n) for all n.

Dominance guarantees that:

- if h2 dominates h1
- and both are admissible

then Astar using h2 will expand no more nodes than Astar using h1. Every node Astar with h2 expands is also expanded by Astar with h1 but not necessarily vice versa.

(c) If both are admissible and h2 dominates, why would you ever use h1?

Computational cost is a pretty good reason. h1 is faster to compute (youre just counting tiles not on goal positions) while h2 requires computing position/difference from goal for every tile. In a time constrained setting or in problems where heuristic computation is a majority of total runtime the simpler heuristic might give better wall clock performance even though it expands more nodes. Also h1 is simpler to explain verify and debug(important if industry your project manager never took an algorithms class).

(d) A third heuristic h3 is proposed: h3 = Manhattan distance + 2 x (number of tiles that must pass through a given centeral square). Evaluate whether h3 is admissible and whether it dominates h2.

Admissibility depends on the puzzle and goal layout. The added penalty 2 x (tiles passing through center) is meant to reflect the constraint that the center area is like a "bottleneck" for certain tiles. Moving tiles across the puzzle is complex and very move intensive. However this is an additional cost added on top of manhattan distance.

So the question is whether the true minimum cost actually accounts for at least 2 extra moves per tile passing through the center.

In many cases h3 is inadmissible. manhattan distance already counts the moves needed for each tiles traversal including moves through the center if thats its shortest route. Adding 2 x (tiles passing through center) on top of manhattan double counts it basically overestimating the cost.

let me propose a specific counterexample to illustrate what I mean by overestimating. If only 1 tile needs to pass through the central square and its manhattan distance is exactly 3, h3 would estimate 3 + 2 = 5, but the actual move count for that tile is 3.

On the topic of dominace, since h3 = h2 + (some non-negative term) h3(n) >= h2(n). So h3 numerically dominates h2. That being said, only h2 is guaranteed admissible. An inadmissible heuristic that "dominates" doesnt help because it can cause Astar to return suboptimal paths. So… h3 is generally inadmissible and shouldnt be used with Astar if optimality matters.

Q20: Musty the mustang is designing an A-star heuristic for the pancake sorting problem: given a stack of N pancakes of distinct sizes, you can insert a spatula at any position and flip all pancakes above it(See figure 3 below). The goal is to sort the stack largest on bottom to smallest on top.

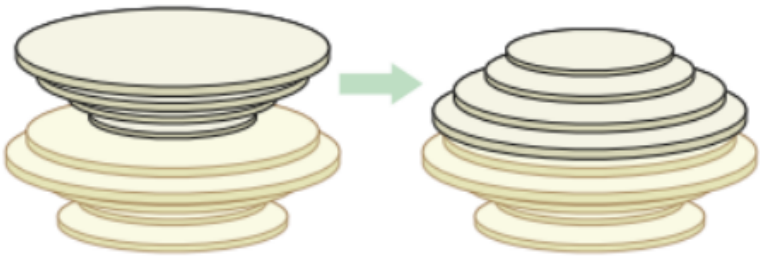

Figure 3: An example of a flip

Musty proposes: h = number of pancakes not in their final position ÷ 2, arguing "each flip can move the spatula through at least 2 pancakes, so we fix at least 2 pancakes per flip."

(a) Identify the flaw in Musty's admissibility argument.

Musty claims that "each flip can move the spatula through at least 2 pancakes, so we fix at least 2 pancakes per flip." This conflates moving pancakes with fixing them.

Importantly, a flip moves multiple pancakes at once however most of those pancakes dont end up in their correct final position. In fact many end up in worse positions. A single flip is more likley to fix a small number of pancakes and may even fix no pancakes(Consider, in some cases you may make a move where non are fixed inorder to set up a better position for future flips). Dividing the count of misplaced pancakes by 2 assumes a fixed rate of progress that the operation simply can not guarantee.

(b) Construct a concrete counterexample with 4 pancakes showing the heuristic either overestimates or that their reasoning breaks down.

Consider the stack [2, 4, 1, 3] (top to bottom), where the goal is [1, 2, 3, 4] (smallest on top, largest on bottom).

All 4 pancakes are out of place so Musty's heuristic gives h = 4 / 2 = 2.

But the optimal solution requires more than 2 flips. Feel free to try it. No matter which flip you make first very few if any pancakes end up in their final positions and you need additional flips to fix the rest. The optimal solution for this stack is 4 flips so the true cost is 4, but h estimates 2, which is fine for admissibility (h <= hstar). The issue is Mustys reasoning is flawed even though the resulting number happens to be admissible.

To prove admissibility, you need a real argument tying h to a lower bound on the optimal cost.

(c) Propose a correct admissible heuristic and prove it is admissible.

Heres one but there are certainly more:

h(n) = number of adjacent pairs in the stack that are not consecutive in the goal ordering(ie: count pairs (i, i+1) in the current stack where the two pancakes at positions i and i+1 are not consecutive in size either ascending or descending).

Each flip can break or fix at most 1 such adjacent gap (specifically, the gap at the position where the spatula was inserted). Other adjacencies in the flipped portion are preserved (just reversed). So each flip reduces the gap count by at most 1, meaning the number of remaining gaps is a lower bound on the number of flips needed. h(n) <= hstar(n).

Consider: h([1,3,2,4]) = 2

[1,3,2,4] insert between 2 and 4

[2,3,1,4] insert between 3 and 1

[3,2,1,4] insert between 1 and 4

[1,2,3,4] Done!

true flips = 3 , heuristic = 2

2<=3 huzzah!

(d) Is your heuristic in (c) more or less informed than the Musty's? Does a more informed heuristic always mean better performance in practice? Explain.

The gap heuristic is more informed than Musty's heuristic as it captures the structural limitations by which pancakes are adjacent and need to be separated as opposed to just counting misplaced pancakes. Moreover, since Musty's heuristic is actually inadmissible (see the example [4,3,2,1] requires 1 flip but h=2), the gap heuristic is strictly preferable for use with Astar when optimality matters.

That being said, more informed does not always mean better in practice. More informed heuristics typically expand fewer nodes, but:

- They may take longer per node to compute.
- Their advantage shrinks on small problems.
- A heuristic thats marginally more informed but much more expensive can be a net loss.

In the pancake problem the gap heuristic is cheap (O(n) per evaluation) and much more informed than misplaced/2, so its clearly a win. In general informedness reduces nodes expanded, but total runtime depends on both nodes expanded and per node cost.

IV Part 4: Adversarial Search

Q21: Define the following terms in the context of adversarial search: [13]

[13] AIMA Chapter 6.1 Page 193

(a) Zero sum game

A game where the total utility across all players adds to a constant (usually zero) for every outcome

ie: one players gain is exactly another players loss

Examples: chess (win/loss/draw) or poker (money won = money lost)

(b) Terminal state

A state where the game is over and no further moves are possible. Terminal states have a defined utility value assigned by the games rules.

(c) Utility function

A function that assigns a numerical value to terminal states representing the how good that outcome is from a players perspective. MAX tries to reach high utility and MIN tries to force low utility.

Q22: Explain why Minimax is equivalent to choosing the optimal strategy in a two player zero sum game where both players play perfectly. What assumption about the opponent makes Minimax appropriate? [14]

[14] AIMA Chapter 6.2 Page 194

Minimax assumes the opponent plays optimally in that at every MIN node the opponent will select the move that minimizes MAXs utility. So in a zero sum game the opponents optimal play directly minimizes your utility (since their gain is your loss). If both players play perfectly the Minimax value of the root represents the best outcome MAX can guarantee against any opponent strategy.

The key assumption is that the opponent is rational and adversarial. So they know the game tree and they play perfectly and their goal is directly opposed to yours. If this assumption holds then Minimax produces the optimal strategy. If the opponent is suboptimal or stochastic Minimax is overly pessimistic and may miss exploitable opportunities.

Q23: Compare and contrast Minimax and Expectimax. When would you choose Expectimax over Minimax, and what changes in the game tree structure? [15]

[15] AIMA Chapter 6.5 Page 211

Minimax assumes the opponent plays optimally (so they chose the worse options for you), so MIN nodes take the minimum of children. Expectimax replaces MIN nodes with CHANCE nodes that compute the weighted average (expected value) of children according to some probability distribution.

So you Choose Expectimax when:

- The opponent is stochastic or imperfect (ie: a random move generator or a beginner player)
- The environment has randomness (ie: dice rolls or card draws)
- You have a probabilistic model of how an outcome will unfold rather than an adversarial one

In the tree MIN nodes become CHANCE nodes. Instead of selecting the worst child you compute an expectation over the children weighted by their probabilities. The values that propagate up are no longer guaranteed minimums or maximums instead theyre expected values. Importantly, this means a single high utility branch can dominate even if it has a risky low utility branch beside it.

For Questions 24-25, use the following game tree.

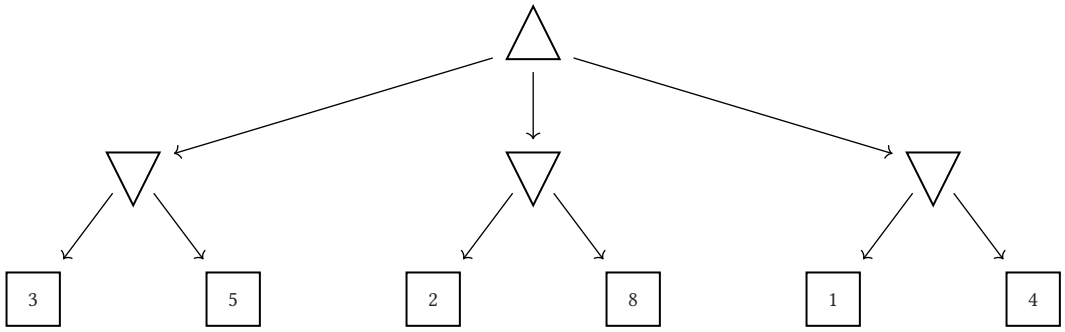

Figure 4: Game Tree for Q24-25

Q24:Compute the Minimax value of the root.

Left MIN: min(3, 5) = 3.

Middle MIN: min(2, 8) = 2.

Right MIN: min(1, 4) = 1.

MAX root: max(3, 2, 1) = 3.

MAX picks the left most branch.

Q25: Using the same tree, apply Alpha Beta Pruning. List which leaf nodes are pruned (not evaluated). Assume children are evaluated left to right.

Start at MAX root: $\alpha = -\infty$, $\beta = +\infty$.

- Left MIN subtree($\alpha = -\infty$, $\beta = +\infty$ passed down):
  - Evaluate leaf 3. MIN updates its $\beta = \min(+\infty, 3) = 3$.
  - Evaluate leaf 5. MIN updates its $\beta = \min(3, 5) = 3$.
  - MIN returns 3. Back at MAX root: $\alpha = \max(-\infty, 3) = 3$.
- Middle MIN subtree ($\alpha = 3$, $\beta = +\infty$ passed down):
  - Evaluate leaf 2. MIN updates its $\beta = \min(+\infty, 2) = 2$.
  - Now β (2) $\leq \alpha$ (3): prune. Leaf 8 is never evaluated.
  - MIN returns ≤ 2. Root α stays 3 (2 is not better for MAX).
- Right MIN subtree ($\alpha = 3$, $\beta = +\infty$ passed down):
  - Evaluate leaf 1. MIN updates its $\beta = \min(+\infty, 1) = 1$.
  - Now β (1) $\leq \alpha$ (3): prune. Leaf 4 is never evaluated.
  - MIN returns ≤ 1. Root α stays 3.

Pruned leaves: 8 and 4. Final Minimax value = 3 (unchanged from full Minimax as pruning never affects correctness).

For Question 26, use the following game tree.

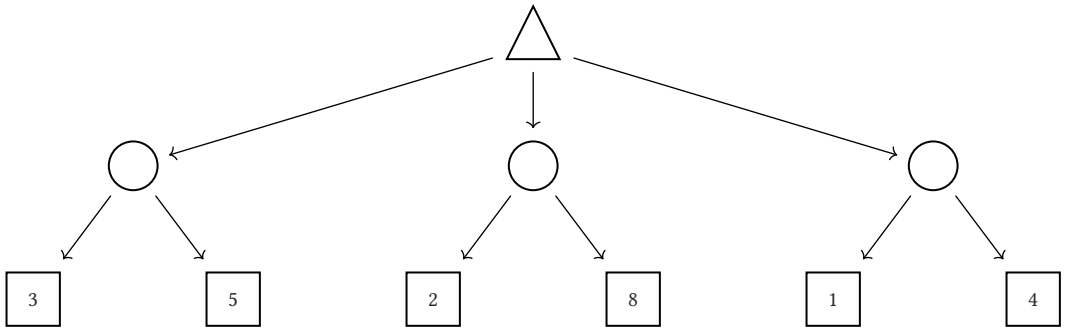

Figure 5: Game Tree for Q26

Q26: Now the second layer nodes are CHANCE nodes (with uniform probability over their children) instead of MIN nodes. Compute the Expectimax value of the root.

Treating second layer nodes as CHANCE nodes with uniform probability (each child weighted 0.5):

- Left CHANCE: (3 + 5) / 2 = 4.0
- Middle CHANCE: (2 + 8) / 2 = 5.0
- Right CHANCE: (1 + 4) / 2 = 2.5

MAX root: max(4.0, 5.0, 2.5) = 5.0. MAX picks the middle branch.

Importantly Expectimax picks the middle branch while Minimax and Alpha-Beta pick the left branch on the same tree. Under Minimax, the middle branch looked terrible because the opponent would force you to the 2. Under Expectimax, the middle branch has the highest expected value because the 8 balances out the 2.

This shows the big difference between these algorithms as Minimax assumes the worst case, Expectimax assumes outcomes average out. Notably how they average out can differ based on weightings. Lets propose an alternate tree were the 2 is weighted 0.8 and the 8 is weighted 0.2:

- Middle CHANCE: 0.8 x 2 + 0.2 x 8 = 1.6 + 1.6 = 3.2

MAX root: max(4.0, 3.2, 2.5) = 4.0. Now Expectimax also picks the left branch, agreeing with Minimax.

When the high value outcome (8) is sufficiently unlikely, the expected value of the middle branch drops below the average 4.0 from the left branch, and both algorithms converge on the same choice for different reasons. Minimax avoids the worst case (2), Expectimax sees that the average outcome is worse when the 2 is highly probable.

Q27: In Alpha Beta pruning, the order in which children are evaluated matters for efficiency but not for correctness. Explain why the final Minimax value is always the same regardless of evaluation order, and describe what child ordering maximizes the number of pruned nodes.

Alpha-Beta pruning only eliminates subtrees that provably can not affect the final Minimax value. The α and β bounds guarantee that any pruned subtree would return a value that is irrelevant to the parent's decision, so the parent has already found a child that's at least as good (or as bad). So no matter what evaluation order is used, the pruned subtrees couldn't have changed the answer, and the result is always identical to full Minimax.

So the order that maxinimzes the number of pruned nodes places the best move first at each node. So the highest valued children first at MAX nodes, and lowest valued children first at MIN nodes. This establishes tight α/β bounds as early as possible, allowing maximum pruning of remaining children. With perfect ordering, the time complexity drops from O(b^d) to O(b^(d/2)), effectively letting you search twice as deep for the same computational budget.

In practice, move ordering heuristics (like trying captures first in chess) approximate this ideal(this could be important for your lab :P)

Q28: Musty is building a chess AI. They argue: "Since Expectimax accounts for the possibility that the opponent plays suboptimally, it is better than Minimax." Is this claim always true, sometimes true, or always false? Provide a brief justification or counterexample.

It is sometimes true becasue Expectimax is better when the opponent really is suboptimal or stochastic. This is because it accurately models a random or imperfect opponent and exploits their mistakes. But it's worse when facing a strong adversary, because Expectimax may choose risky moves that an optimal opponent would punish.

Take chess specifically, the claim is just wrong. Serious chess opponents play near optimally[16] so modeling them as random or stochastic would lead to overconfident, exploitable strategies. The standard approach in chess AI is Minimax with alpha-beta pruning, exactly because the worst case (adversarial) assumption is appropriate against strong opponents. Expectimax would actually be worse for chess in most realistic settings.

[16] https://www.vice.com/en/article/did-hans-neimann-cheat-at-chess-with-a-sex-toy-this-coder-is-attempting-to-find-out/

The right choice depends on your opponent model. Use Minimax when the opponent is adversarial and use Expectimax when the opponent is random, stochastic, or suboptimal in known ways. "Always better" is too strong a claim for either.

For Question 29, use the following game tree.

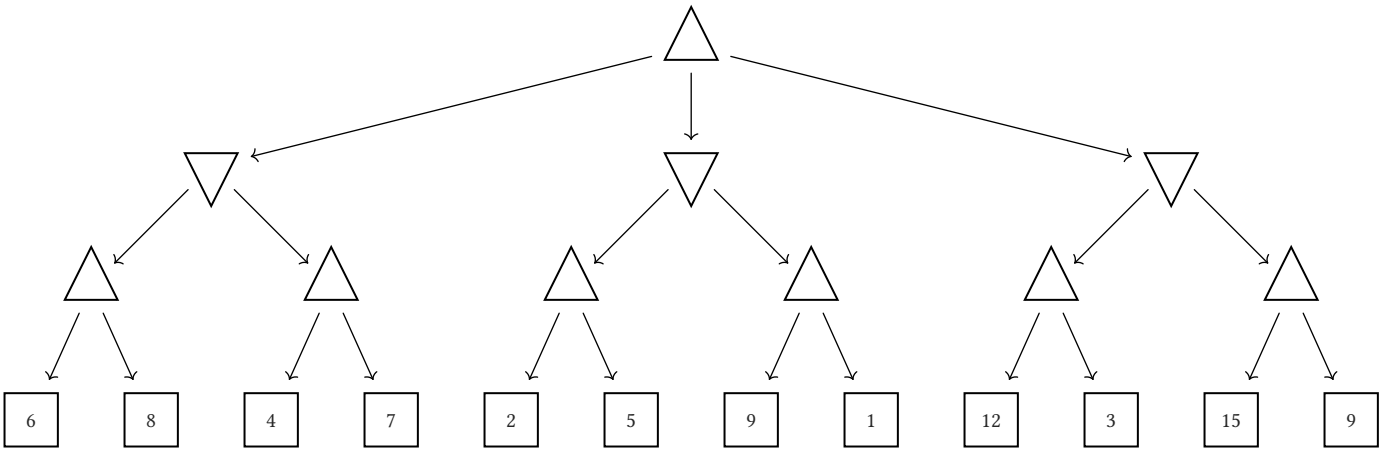

Figure 6: Game Tree for Q29

Q29: Using the above tree, apply Alpha Beta Pruning. List which leaf nodes are pruned (not evaluated). Assume children are evaluated left to right.

Start at MAX root: $\alpha = -\infty$, $\beta = +\infty$.

- Left MIN subtree ($\alpha = -\infty$, $\beta = +\infty$ passed down):
  - Left MAX child ($\alpha = -\infty$, $\beta = +\infty$):
    - Evaluate leaf 6. $\alpha = \max(-\infty, 6) = 6$.
    - Evaluate leaf 8. $\alpha = \max(6, 8) = 8$.
    - Returns 8.
  - Left MIN updates $\beta = \min(+\infty, 8) = 8$.
  - Right MAX child ($\alpha = -\infty$, $\beta = 8$):
    - Evaluate leaf 4. $\alpha = \max(-\infty, 4) = 4$. No pruning.
    - Evaluate leaf 7. $\alpha = \max(4, 7) = 7$. No pruning ($7 < 8$).
    - Returns 7.
  - Left MIN updates $\beta = \min(8, 7) = 7$.
  - Left MIN returns 7. Back at MAX root: $\alpha = \max(-\infty, 7) = 7$.
- Middle MIN subtree ($\alpha = 7$, $\beta = +\infty$ passed down):
  - Left MAX child ($\alpha = 7$, $\beta = +\infty$):
    - Evaluate leaf 2. $\alpha = \max(7, 2) = 7$. No pruning.
    - Evaluate leaf 5. $\alpha = \max(7, 5) = 7$. No pruning.
    - Returns 5.
  - Middle MIN updates $\beta = \min(+\infty, 5) = 5$.
  - Now β (5) $\leq \alpha$ (7): prune. The entire right MAX child (leaves 9 and 1) is never evaluated.
  - Middle MIN returns ≤ 5. Root α stays 7.
- Right MIN subtree ($\alpha = 7$, $\beta = +\infty$ passed down):
  - Left MAX child ($\alpha = 7$, $\beta = +\infty$):
    - Evaluate leaf 12. $\alpha = \max(7, 12) = 12$. No pruning.
    - Evaluate leaf 3. $\alpha = \max(12, 3) = 12$. No pruning.
    - Returns 12.
  - Right MIN updates $\beta = \min(+\infty, 12) = 12$.

- Right MAX child (α = 7, β = 12):
  - Evaluate leaf 15. α = max(7, 15) = 15.
  - Now α (15) $\geq$ β (12): prune. Leaf 9 is never evaluated.
  - Returns $\geq$ 15.
- Right MIN returns min(12, $\geq$ 15) = 12. Root α = max(7, 12) = 12.

Pruned leaves: 9 and 1 (right MAX subtree of Middle MIN), and 9 (second child of Right MAX of Right MIN). Final Minimax value = 12 (unchanged from full Minimax).